

TEAM BASED PROJECT

Smart Library Management & Recommendation System

Topik 6 | Queue · Linked list · Stack · BST · Graph · Sorting

Kelompok : 6

Danar Hafidz A'ray · Anargya Keano · Itqon El Fikry Fausta

Fauzi Fajar Wijaya · Muhammad Daffaa Adhirajasa

Prodi Teknik Elektro — Universitas Negeri Yogyakarta | 2026

Latar Belakang & Masalah

UNY Library melayani ribuan anggota setiap hari. Terdapat tiga masalah utama yang harus diselesaikan:



Pengelolaan sistem antrian

Bertujuan agar:

1. Urutan transaksi tidak kacau
2. Sistem berjalan secara adil

Solusi: Implementasi Queue (FIFO)



Kesalahan Input Petugas

Dibutuhkan fitur UNDO
untuk membatalkan
transaksi terakhir.

Solusi: Implementasi (undo)



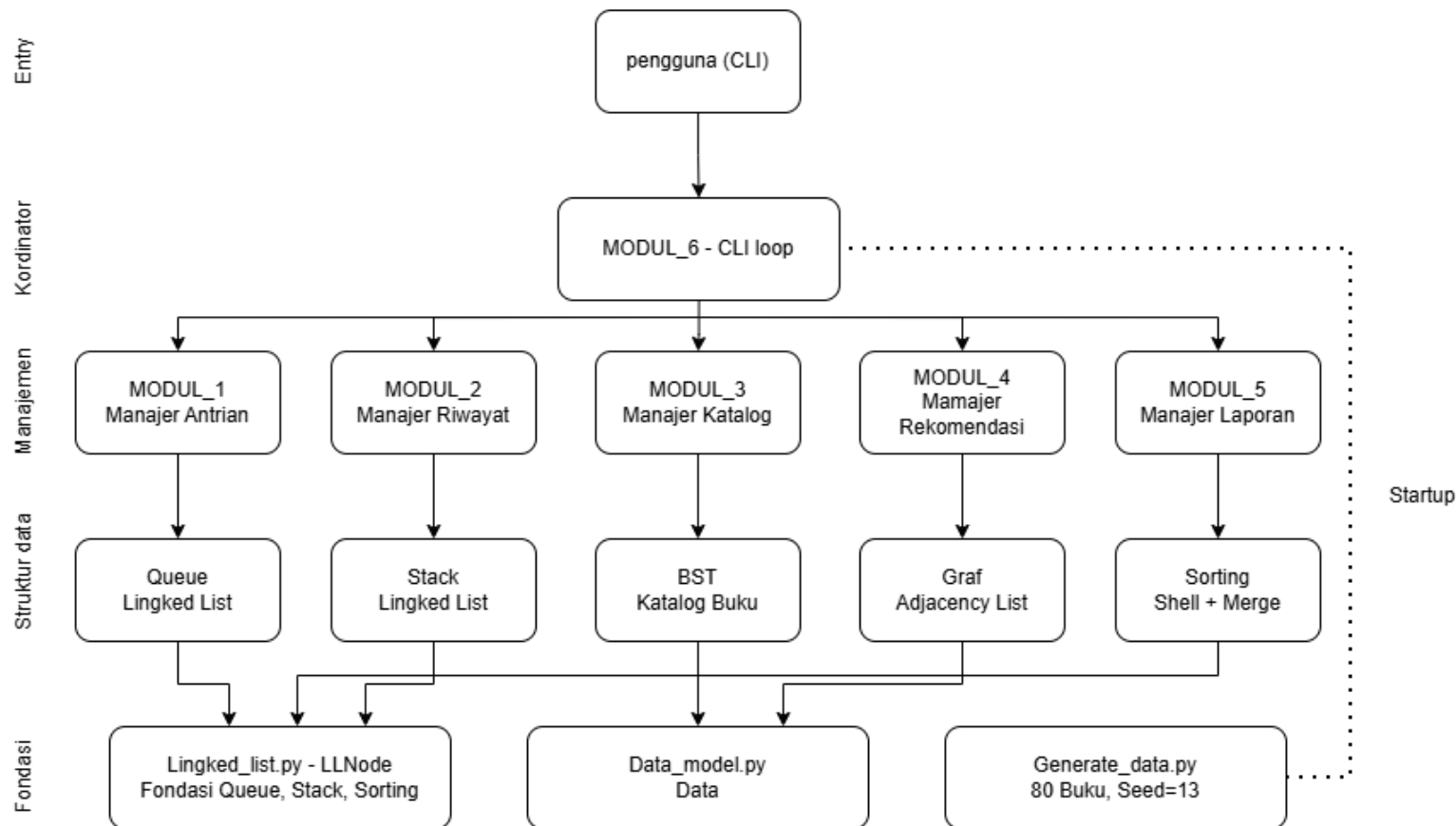
Rekomendasi Buku

Fitur rekomendasi berbasis data pola
peminjaman

Solusi: Implementasi Graph (ko-
pinjam) + BFS

Solusi: Sistem perpustakaan digital CLI mengintegrasikan Queue (FIFO), Stack (undo), BST, Graph & Sorting dari nol.

Arsitektur Sistem — 6 Modul Terintegrasi



Queue & Stack — Linked List dari Nol

Queue — Antrian Pemesanan (FIFO)

enqueue / dequeue: $O(1)$

- Pointer tail $\rightarrow$ enqueue tanpa traversal
- Dequeue langsung cabut head
- 1 Queue per ISBN (dict)

batalan_pesana: $O(k)$ rebuild

Ruang: $O(n)$

NIM-001

NIM-002

NIM-003

HEAD (dequeue)

TAIL (enqueue)

Antrian ISBN-0001 — 3 anggota mengantri

Stack — Riwayat Transaksi (LIFO)

push / pop: $O(1)$

- top = head Linked List
- BATALKAN_TERAKHIR $\rightarrow$ pop
- Archiving setelah N transaksi

Ruang: $O(n)$

TX-003 PINJAM $\leftarrow$ TOP

TX-002 KEMBALIKAN

TX-001 PINJAM

UNDO pop TX-003 $\rightarrow$ status buku $\rightarrow$ TERSEDIA

BST — Katalog Buku

Kunci = ISBN string (leksikografis) | $O(\log n)$ rata-rata | $O(n)$ worst-case (data terurut)

insert()

Rekursif. Return bool True/False — bedakan node baru vs duplikat agar counter tidak salah naik

search()

Bandingkan ISBN di tiap node, belok kiri/kanan. $O(\log n)$ avg — $O(n)$ worst jika pohon miring

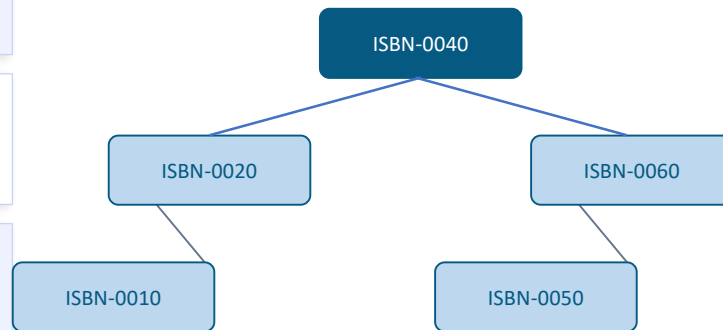
inorder()

kiri → root → kanan. Hasilkan katalog terurut ISBN otomatis. $O(n)$

Penjelasan singkat

BST berfungsi untuk menyimpan katalog buku dan berdampak untuk search ISBN (cepat). Mendukung range query.

BST: data acak vs terurut



Data acak (seed=13): tinggi = $\log_2(n) = 12$

Data terurut: tinggi = $n = 80$ (pohon miring!)

Empiris $N=300$:

acak = 16 level vs terurut = 300 level

Graph — Rekomendasi Ko-pinjam (BFS)

Graf tak-berarah berbobot | Adjacency List | BFS max 2 hop | Big-O: $O(V+E)$

Penjelasan

Berfungsi untuk merekomendasikan buku berdasarkan ko-pinjam (bobot edge). BFS bekerja dengan cara menjelajah perlevel (hop 1, hop 2)

Kenapa adjacency list

Karena sparse graph (sisi sedikit di banding maks. koneksi)

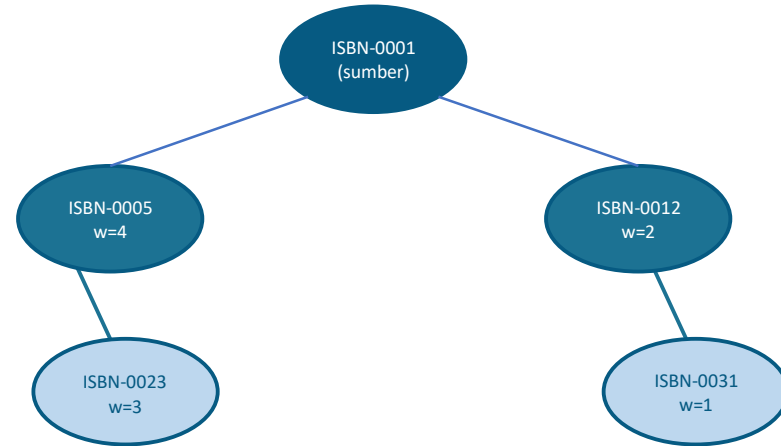
rekomendasikan()

BFS pakai Queue LL sendiri (bukan deque). max_hop=2, filter min_bobot.
Sort hasil: insertion sort manual

Sesi Ko-pinjam

Setiap PINJAM baru dipasangkan dengan semua buku aktif anggota → edge bobot naik otomatis

BFS 2 Hop dari ISBN-0001:



Hop 1 (biru tua) | Hop 2 (biru muda)
BFS pakai Queue LL sendiri — bukan deque!

Sorting Laporan Bulanan — Shell Sort & Merge Sort

Keduanya diimplementasikan langsung pada Linked List — tanpa konversi ke array Python

Shell Sort $\sim O(n^{1.5})$

Kegunaan: Sorting durasi peminjaman (desc)

Gap Knuth: 1, 4, 13, 40 ...
(gap = gap $\times$ 3+1)

Teknik LL: node linked list di kumpulkan ke array pointer,
sort dengan swap DATA bukan pointer

N=20: 0.000046s N=80: 0.000301s
N=300: 0.002506s

Merge Sort $O(n \log n)$

Kegunaan: Sorting frekuensi per ISBN (desc)

kelebihan: Lebih stabil dan cepat untuk data besar
split: Slow/fast pointer temukan
tengah LL dalam $O(n)$

Merge: Sentinel node dummy — hindari
penanganan head khusus

N=20: 0.000039s N=80: 0.000166s
N=300: 0.000357s (7× lebih cepat!)

Demo CLI — Skenario Uji End-to-End

Setiap perintah menampilkan Big-O operasinya | Seed=13 menjamin hasil reprodusibel

```
>> PINJAM NIM-001 ISBN-0001
[PINJAM] Berhasil. NIM-001 meminjam ISBN-0001 selama 14 hari. TX-0001
Big-O:  $O(\log n)$  BST search + update
>> PESAN NIM-002 ISBN-0001
[ANTRIAN] NIM-002 berhasil masuk antrian ISBN-0001. Posisi ke-1.
Big-O: enqueue  $O(1)$ 
>> REKOMENDASI ISBN-0001
[REKOMENDASI] Buku yang sering dipinjam bersama ISBN-0001:
(belum ada data ko-pinjam untuk buku ini)
Big-O:  $O(V+E)$  BFS graf ko-pinjam
>> BATALKAN_TERAKHIR
[UNDO] Membatalkan: [TX-0002] PESAN | NIM: NIM-002 | ISBN: ISBN-0001 | 2026-05-25 14:21:42
[UNDO] Pesanan NIM-002 untuk ISBN-0001 dihapus dari antrian.
Big-O:  $O(1)$  stack pop +  $O(\log n)$  BST update
>> LAPORAN_BULAN

[LAPORAN] Peminjaman Berdasarkan Durasi (Shell Sort, descending):
No TX-ID      NIM      ISBN      Durasi (hari)
-----
1 TX-0001     NIM-001   ISBN-0001   14
Big-O Shell Sort:  $\sim O(n^{1.5})$ 

[LAPORAN] Frekuensi Peminjaman per Judul (Merge Sort, descending):
No ISBN      Frekuensi
-----
1 ISBN-0001   1
Big-O Merge Sort:  $O(n \log n)$ 

[LAPORAN] Tabel Runtime Sorting
N | Shell Sort (s) | Merge Sort (s)
-----
1 | 0.000006 | 0.000002
1 | 0.000003 | 0.000001
1 | 0.000001 | 0.000000
Big-O Shell:  $\sim O(n^{1.5})$  | Big-O Merge:  $O(n \log n)$ 
```

Analisis Big-O — Semua Operasi Utama

Struktur Data	Operasi	Big-O Waktu	Big-O Ruang	Kasus
Queue (LL)	enqueue / dequeue	$O(1) / O(1)$	$O(n)$	Semua
Stack (LL)	push / pop / peek	$O(1) / O(1) / O(1)$	$O(n)$	Semua
Linked List	tambah_depan / belakang	$O(1) / O(1)^*$	$O(n)$	*dengan tail ptr
BST	insert / search / delete	$O(\log n)$	$O(\log n)$	Rata-rata
BST	insert / search / delete	$O(n)$	$O(n)$	Worst-case
BST	inorder traversal	$O(n)$	$O(n)$	Semua
Graph	add_copinjam	$O(\deg)$	$O(V+E)$	Semua
Graph BFS	rekomendasikan	$O(V+E)$	$O(V)$	Semua
Shell Sort (LL)	sorting durasi	$\sim O(n^{1.5})$	$O(n)$ ptr	Gap Knuth
Merge Sort (LL)	sorting frekuensi	$O(n \log n)$	$O(\log n)$	Semua

Hasil Eksperimen Runtime (seed=13, avg 5 ulangan)

Struktur Data / Operasi	N=20 (s)	N=80 (s)	N=300 (s)	Tren
Queue enqueue N elemen	0.000014	0.000035	0.000134	Linear $O(n)$ total
Stack push N elemen	0.000018	0.000060	0.0001218	Linear $O(n)$ total
BST insert acak	0.000104	0.000298	0.001084	$O(n \log n)$ — perlahan
BST insert terurut (worst)	0.000079	0.001016	0.021517	$O(n^2)$ — sangat cepat naik!
BST search (avg)	0.000001	0.000002	0.000005	$O(\log n)$ — hampir konstan
Graph BFS	0.000024	0.000040	0.000122	$O(V+E)$ — lambat naik
Shell Sort (LL)	0.000844	0.001271	0.004370	$\sim O(n^{1.5})$
Merge Sort (LL)	0.000070	0.000299	0.000570	$O(n \log n)$ — lebih cepat!

BST insert terurut: N=20→N=300 naik 107× (konfirmasi worst-case $O(n^2)$) | Merge Sort 7× lebih cepat dari Shell Sort pada N=300

Keputusan Desain & Efisiensi



BST vs Hash Table

BST dipilih karena mendukung inorder traversal otomatis menghasilkan katalog terurut ISBN $O(n)$ tanpa sort tambahan, dan mendukung range query. Hash Table $O(1)$ lebih cepat untuk point query saja.



Queue biasa vs Priority Queue

Queue biasa (FIFO) dipilih untuk menjamin fairness. Priority Queue: enqueue $O(n)$ vs $O(1)$ — 50× lebih lambat untuk 50 anggota antrian.



BFS vs DFS untuk Rekomendasi

BFS menjelajah level per level — mudah dibatasi $\text{max_hop}=2$. DFS sulit dikontrol batasannya. BFS $O(V+E)$ optimal untuk sparse graph rekomendasi.



Merge Sort vs Shell Sort

Merge Sort guaranteed $O(n \log n)$ untuk semua kasus. Shell Sort $\sim O(n^{1.5})$ lebih cepat hanya untuk data hampir terurut. Untuk N besar, Merge Sort 7× lebih unggul.



Adjacency List vs Matrix

Graf buku bersifat sparse — kebanyakan buku tidak terhubung langsung. Adjacency list $O(V+E)$ jauh lebih efisien dari $O(V^2)$ matrix untuk kasus ini.

Kesimpulan

Smart Library Management & Recommendation System

✓ 6 struktur data dari nol: Queue, Stack, BST, Graph, Linked List, Sorting

✓ 117 unit test case lulus — test_queue, test_stack, test_bst, test_graph, test_sorting, test_linked_list

✓ Simulasi 300 event end-to-end selesai dalam 0.0032 detik (~10.7 μ s/event)

✓ 10 perintah CLI — setiap perintah tampilkan Big-O operasinya

✓ Benchmark 3 ukuran data (N=20, 80, 300) — empiris konfirmasi teori Big-O

Kelompok 6 · Prodi Teknik Elektro UNY · ELT60213 Algoritma dan Struktur Data · 2026

Terima Kasih